École Nationale des Sciences Appliquées de Tétouan

Filière: Big Data & Intelligence Artificielle

Trafic Parisien en Temps Réel : Analyse Intelligente avec Kafka, Spark et MongoDB

Une approche basée sur le traitement des données en temps réel

Réalisé par :

- Abdessamad Ait Brahim
- Ayyoub Amazal

Encadré par :

- Dr. Imad sassi

Année Académique : 2024-2025

Remerciement

Nous tenons à exprimer notre profonde gratitude à notre professeur et encadrant, Madame **Imad Sassi**, pour sa disponibilité, sa patience et la richesse de ses conseils tout au long de ce projet.

Son expertise, sa rigueur scientifique et son implication pédagogique ont grandement contribué à l'aboutissement de ce travail. Son accompagnement attentif et ses orientations précieuses nous ont permis de surmonter les obstacles rencontrés et de progresser aussi bien sur le plan technique que personnel.

Ce projet a été pour nous une véritable opportunité d'apprentissage, d'exploration intellectuelle et de collaboration, que nous avons menée avec enthousiasme et détermination, guidés par ses encouragements et sa vision claire.

Nous tenons également à souligner l'importance de son mentorat qui a dépassé le simple cadre académique, nous inculquant des valeurs de persévérance, d'excellence et d'intégrité scientifique qui nous accompagneront bien au-delà de cette formation.

« Comme tous ceux qui résistent patiemment face à notre ignorance,
vous méritez plus que de la reconnaissance... vous méritez notre respect. »

Avec notre sincère reconnaissance,
Abdessamad Ait Brahim et Ayyoub Amazal

Table des matières

1 Résumé **3**

2 Abstract **3**

3 Introduction **4**

4 Objectifs **4**

5 Architecture du Système **5**

6 Préparation des Données **6**
6.1 Source des Données . . . 6
6.2 Attributs Principaux Exploités . . . 6
6.3 Étapes de Prétraitement . . . 6
6.4 Illustration du Pipeline de Préparation . . . 7

7 Modélisation **7**
7.1 Modèles testés et justification . . . 8
7.1.1 Régression Logistique (Logistic Regression) . . . 8
7.1.2 Arbre de Décision (Decision Tree) . . . 8
7.1.3 Forêt Aléatoire (Random Forest) . . . 9
7.2 Métrique d'évaluation : Accuracy . . . 10
7.3 Modèle final retenu . . . 11

8 Prédiction en Temps Réel **12**
8.1 Intégration du modèle dans Spark Streaming . . . 12
8.2 Stockage des résultats prédits . . . 12
8.3 Visualisation dynamique . . . 12

9 Visualisation avec Streamlit **14**
9.1 Carte interactive . . . 14
9.2 Indicateurs Clés de Performance (KPI) . . . 14
9.3 Graphiques temporels . . . 15
9.4 Capture d'écran de l'interface Streamlit . . . 15

10 Résultats et Analyse **15**
10.1 Performances du modèle . . . 16
10.2 Analyse de la visualisation . . . 16
10.3 Extensibilité du système . . . 16
10.4 Illustrations des résultats . . . 17

11 Difficultés Rencontrées **17**
11.1 Compatibilité entre Spark 3.0.0, PySpark et les packages Kafka/MongoDB 18
11.2 Erreurs de parsing des données `geo_point_2d` . . . 18
11.3 Gestion des formats temporels hétérogènes . . . 18

12 Conclusion **19**
12.1 Synthèse du projet 19
12.2 Performances et extensibilité 19
12.3 Perspectives d'amélioration 19

13 Perspectives **20**
13.1 Intégration de données météorologiques 20
13.2 Détection automatique d'anomalies 20
13.3 Déploiement cloud et scalabilité 20
13.4 Autres axes d'amélioration 21

14 Les références **21**

1 Résumé

Ce projet a pour ambition de développer un système intelligent de **prédiction en temps réel de l'état du trafic routier à Paris**, en réponse aux défis croissants liés à la congestion urbaine. Il combine des technologies *Big Data* et des modèles de *Machine Learning* pour collecter, traiter et analyser les données issues de capteurs routiers.

L'architecture s'appuie sur un pipeline complet et modulaire :

- **Apache Kafka** pour l'ingestion des données en continu,
- **Apache Spark** pour le traitement en streaming et l'application des modèles prédictifs,
- **MongoDB** pour le stockage géolocalisé des résultats,
- **Streamlit** pour une visualisation interactive sous forme de carte, indicateurs et graphiques.

Les données exploitées proviennent de la plateforme *Open Data* de la Ville de Paris, notamment du jeu *"Comptages routiers permanents"*. Après un prétraitement rigoureux, plusieurs modèles de classification ont été comparés (régression logistique, arbre de décision, forêt aléatoire). La **régression logistique** a été retenue pour son bon équilibre entre précision et performance.

Le système fournit une vue claire de l'état du trafic en temps réel, aidant à la **prise de décision rapide**, à la **réduction des embouteillages** et à **l'optimisation de la mobilité urbaine**. Il est facilement extensible à d'autres villes et peut intégrer des données complémentaires comme la météo ou les incidents routiers. Ce projet constitue ainsi une avancée concrète vers des **solutions de transport intelligentes**, au service des **villes durables et connectées**.

2 Abstract

This project aims to develop an intelligent system for **real-time prediction of road traffic conditions in Paris**, addressing the growing challenges related to urban congestion. It combines *Big Data* technologies and *Machine Learning* models to collect, process, and analyze data from traffic sensors.

The architecture is based on a complete and modular pipeline :

- **Apache Kafka** for continuous data ingestion,
- **Apache Spark** for streaming processing and applying predictive models,
- **MongoDB** for geolocated storage of results,
- **Streamlit** for interactive visualization including maps, indicators, and graphs.

The data used comes from the *Open Data* platform of the City of Paris, notably the *"Comptages routiers permanents"* dataset. After thorough preprocessing, several classification models were compared (logistic regression, decision tree, random forest). **Logistic regression** was selected for its good balance between accuracy and performance.

The system provides a clear real-time view of traffic conditions, supporting **quick decision-making**, **congestion reduction**, and **optimization of urban mobility**. It is easily extendable to other cities and can integrate complementary data such as weather or traffic incidents. This project thus represents a concrete step towards **intelligent transportation solutions** serving **sustainable and connected cities**.

3 Introduction

Dans les grandes métropoles modernes, comme Paris, la croissance continue du nombre de véhicules engendre une congestion routière croissante, générant des embouteillages fréquents, une augmentation de la pollution atmosphérique et une perte de temps significative pour les usagers. Cette situation pose un défi important pour les autorités et les gestionnaires de la mobilité urbaine, qui doivent constamment chercher des solutions efficaces pour fluidifier le trafic et améliorer la qualité de vie des habitants.

La capacité à prédire l'état du trafic en temps réel est devenue un enjeu stratégique majeur. En anticipant les fluctuations du trafic, il est possible de mettre en place des mesures proactives, telles que la réorganisation dynamique des feux de signalisation, l'orientation du trafic vers des itinéraires alternatifs, ou encore la diffusion d'informations précises aux usagers via des applications mobiles. Ces actions contribuent à réduire les embouteillages, diminuer la consommation de carburant et limiter l'impact environnemental.

Ce projet s'inscrit dans ce contexte en proposant de développer un système intelligent capable de surveiller en continu le trafic routier parisien et de fournir des prévisions précises et actualisées. Pour cela, nous combinons les technologies de *Big Data* — qui permettent de collecter et traiter d'importants volumes de données en temps réel — et des modèles avancés de *Machine Learning*, qui exploitent ces données pour apprendre les comportements du trafic et anticiper son évolution.

L'objectif est de concevoir un pipeline complet allant de la collecte des données issues des capteurs routiers, à leur traitement en flux continu, jusqu'à la production de prédictions et leur visualisation via une interface interactive. Ce système vise à offrir un outil performant et scalable, directement exploitable par les autorités ou les applications de mobilité urbaine pour améliorer la gestion des flux et la planification des déplacements.

4 Objectifs

Ce projet a pour principaux objectifs de :

- **Collecter et traiter les données de trafic en temps réel** : Exploiter les flux continus de données provenant des capteurs routiers installés à Paris, en utilisant des technologies Big Data comme Apache Kafka pour l'ingestion des données et Apache Spark pour leur traitement en streaming. Cette étape permet de garantir la disponibilité immédiate des informations nécessaires pour une analyse dynamique du trafic.
- **Construire un pipeline de prédiction performant basé sur des modèles de Machine Learning** : Mettre en place et entraîner plusieurs modèles de classification (régression logistique, arbres de décision, forêts aléatoires) capables d'analyser les données de trafic en temps réel afin d'anticiper l'état du trafic (fluide, pré-saturé, saturé, bloqué). Le pipeline doit intégrer le chargement du modèle dans le processus de traitement en streaming pour effectuer des prédictions instantanées.
- **Visualiser l'état du trafic prédit à l'aide d'un tableau de bord interactif** : Développer une interface conviviale et dynamique avec *Streamlit* permettant aux utilisateurs de consulter en temps réel la carte des zones de trafic, des indicateurs clés (KPI) sur l'état global du réseau routier, ainsi que des graphiques temporels illustrant l'évolution des conditions de circulation. Cette visualisation facilite la prise de décision rapide et informée par les autorités ou les citoyens.

5 Architecture du Système

L'architecture du système est conçue pour assurer un traitement fluide, scalable et en temps réel des données de trafic. Elle se compose de plusieurs composants intégrés, chacun jouant un rôle spécifique :

- **Apache Kafka** : Sert de système d'ingestion en temps réel, Kafka collecte les données JSON générées par les capteurs routiers et les diffuse sous forme de flux. Cette architecture de messagerie distribuée garantit une transmission rapide et fiable des données à travers le pipeline.
- **Apache Spark Structured Streaming** : Utilisé pour le traitement en continu des données reçues via Kafka, Spark réalise le nettoyage des données (suppression des valeurs manquantes, transformation des formats, parsing JSON) et prépare ces données pour la phase de prédiction. Son moteur de traitement distribué permet d'assurer une faible latence et une grande robustesse.
- **Spark MLlib** : Intégré à Spark, ce module de Machine Learning applique plusieurs modèles de classification tels que la *régression logistique*, l'arbre de décision et la forêt aléatoire pour prédire en temps réel l'état du trafic. Les modèles sont chargés dans le pipeline streaming et évalués en continu.
- **MongoDB** : Les résultats des prédictions, associés aux coordonnées géographiques des capteurs, sont stockés dans une base NoSQL MongoDB. Cette base permet un accès rapide et flexible aux données historiques et en temps réel pour la visualisation et l'analyse.
- **Streamlit** : Une interface utilisateur interactive est développée avec Streamlit, affichant les données sur une carte dynamique, des indicateurs clés de performance (KPI) et des graphiques temporels. Cette application web facilite la consultation en temps réel et la prise de décision.

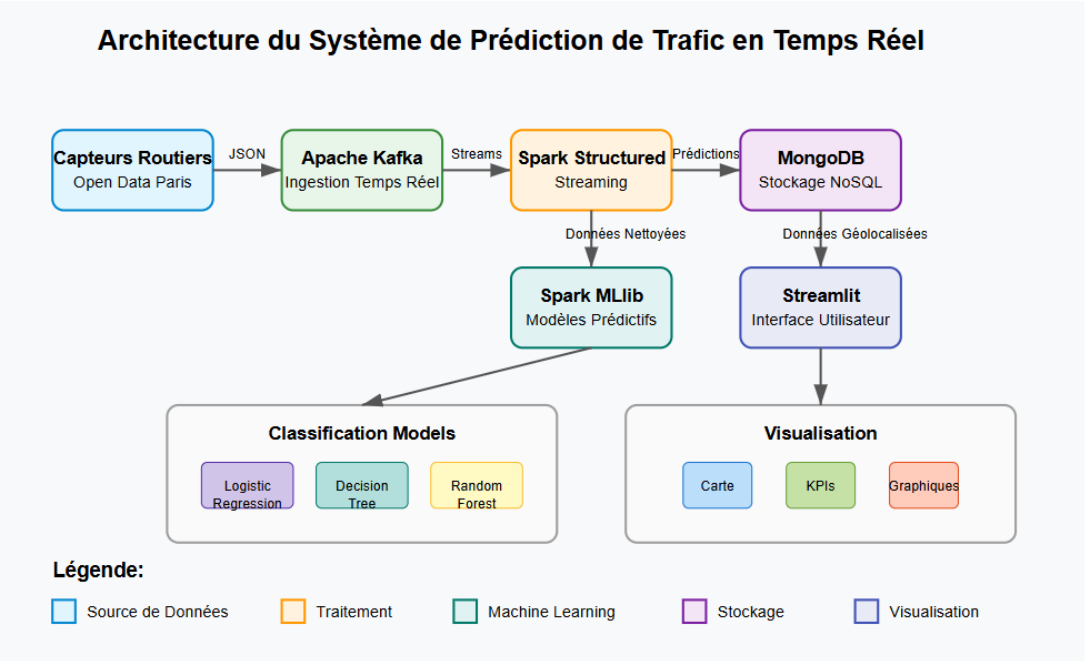


FIGURE 1 – Schéma de l'architecture du système de prédiction du trafic routier

6 Préparation des Données

6.1 Source des Données

Les données utilisées dans ce projet proviennent de la plateforme officielle **Open Data Paris**, accessible à l'adresse suivante :

- https://opendata.paris.fr/explore/dataset/comptage-routier-permanent/

Le jeu de données intitulé *Comptages routiers permanents* fournit des mesures horaires collectées par des capteurs inductifs installés dans les voiries parisiennes. Il offre une couverture spatio-temporelle importante avec des données régulièrement mises à jour, incluant les caractéristiques du trafic telles que le débit, la densité et des informations géographiques précises.

6.2 Attributs Principaux Exploités

Pour répondre aux objectifs de prédiction du trafic, nous avons extrait les champs suivants :

- **q** : Débit de véhicules (nombre de véhicules passés en une heure).
- **k** : Densité de circulation (occupation de la voie).
- **t_1h** : Horodatage (timestamp à l'heure).
- **etat_trafic** : Étiquette de classification représentant l'état du trafic (*fluide, présaturé, saturé, bloqué*).
- **geo_point_2d** : Coordonnées GPS du capteur.

6.3 Étapes de Prétraitement

Le prétraitement constitue une étape cruciale pour transformer les données brutes en un format exploitable pour le modèle de Machine Learning. Voici les étapes réalisées :

- **Chargement et parsing des fichiers JSON** :
  - Utilisation de Spark Structured Streaming pour lire les flux de données JSON en temps réel.
  - Extraction automatique des champs imbriqués et conversion vers une structure tabulaire.
- **Nettoyage des données** :
  - Suppression des lignes avec des valeurs manquantes ou nulles dans les colonnes clés (`q`, `k`, `etat_trafic`).
  - Élimination des doublons et des valeurs extrêmes (e.g., débit anormalement élevé).
- **Transformation géographique** :
  - Séparation du champ `geo_point_2d` en deux colonnes distinctes : `latitude` et `longitude`.

- Encodage compatible avec MongoDB pour un stockage géolocalisé et une visualisation cartographique.

- **Encodage des labels** :
  - Transformation de la variable `etat_trafic` en classes numériques (e.g., 0 : fluide, 1 : présaturé, 2 : saturé, 3 : bloqué).
- **Normalisation des variables numériques** (optionnelle selon le modèle) :
  — Application d'un *Min-Max Scaling* ou d'une *standardisation* (moyenne centrée et écart-type unitaire) sur `q` et `k`.

6.4 Illustration du Pipeline de Préparation

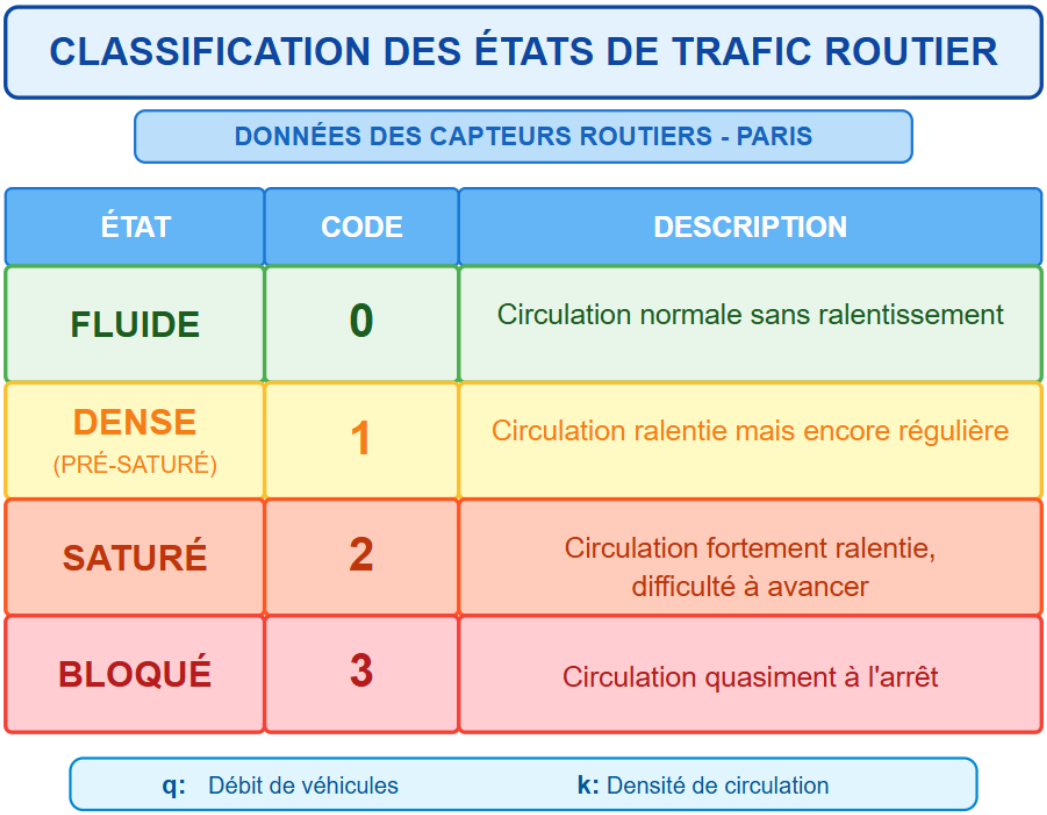


FIGURE 2 – Pipeline de préparation des données depuis la source JSON jusqu'au jeu prêt pour l'entraînement.

7 Modélisation

La tâche de prédiction de l'état du trafic peut être formulée comme un problème de classification supervisée, dans lequel l'objectif est de prédire une variable catégorielle `etat_trafic`, représentant l'état du trafic (`fluide`, `ralenti`, `congestionné`), à partir d'un ensemble de variables explicatives (`q`, `k`, `t_1h`, etc.).

Pour cette étude, nous avons sélectionné trois modèles de classification classiques mais puissants : **Régression Logistique**, **Arbre de Décision**, et **Forêt Aléatoire**. Ces modèles sont intégrés dans la bibliothèque `SparkML`, ce qui permet un apprentissage distribué et scalable sur de gros volumes de données.

7.1 Modèles testés et justification

7.1.1 Régression Logistique (Logistic Regression)

La régression logistique est un modèle linéaire de classification qui estime la probabilité d'appartenance à une classe via la fonction logistique (sigmoïde). Pour un vecteur d'attributs $x = (x_1, x_2, ..., x_n)$, le modèle s'écrit :

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n)}}$$

Dans le cas multi-classes, on utilise une généralisation appelée **softmax** :

$$P(y = k|x) = \frac{e^{\beta_k^T x}}{\sum_{j=1}^{K} e^{\beta_j^T x}}$$

Avantages :

— Très rapide à entraîner et à inférer.

— Facilement interprétable grâce aux coefficients β_i.

— Robuste et stable pour des flux de données en temps réel.

— Faible consommation de mémoire et de ressources.

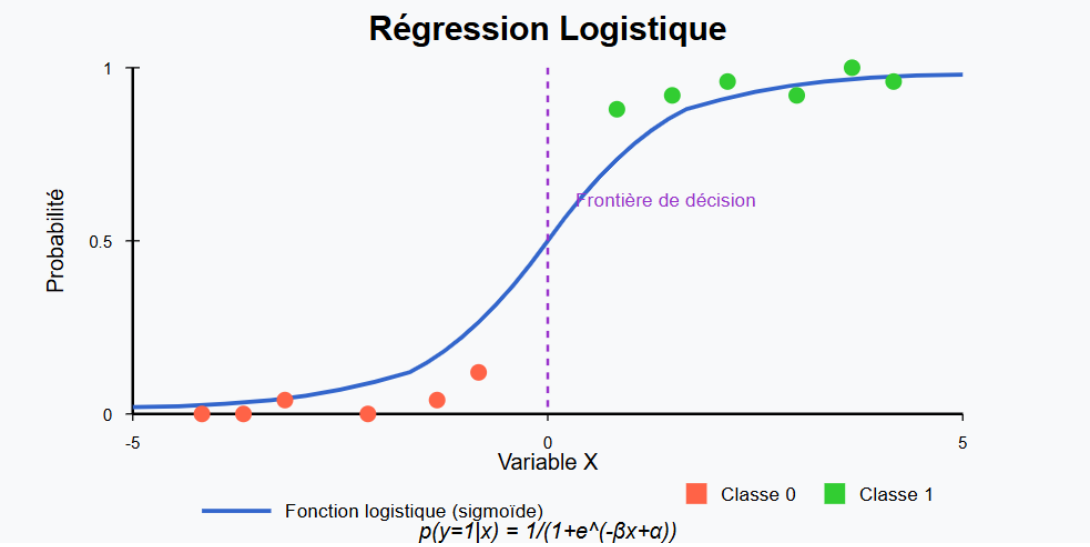


FIGURE 3 – Modélisation de Régression logistique

7.1.2 Arbre de Décision (Decision Tree)

L'arbre de décision est un modèle non linéaire basé sur une structure arborescente. À chaque nœud, une règle de décision est appliquée sur une caractéristique (par exemple $x_i < \theta$) pour diviser l'espace des données. Le modèle cherche à maximiser la pureté des feuilles via des métriques comme :

- **Indice de Gini** : $G = 1 - \sum_{i=1}^{K} p_i^2$ - **Entropie** : $H = -\sum_{i=1}^{K} p_i \log(p_i)$

Avantages :

— Capable de modéliser des relations complexes et non linéaires.
— Intuitif et visuellement interprétable.
— Aucune hypothèse sur la distribution des données.

Inconvénients :

— Forte sensibilité au bruit.
— Risque d'overfitting si l'arbre est trop profond.

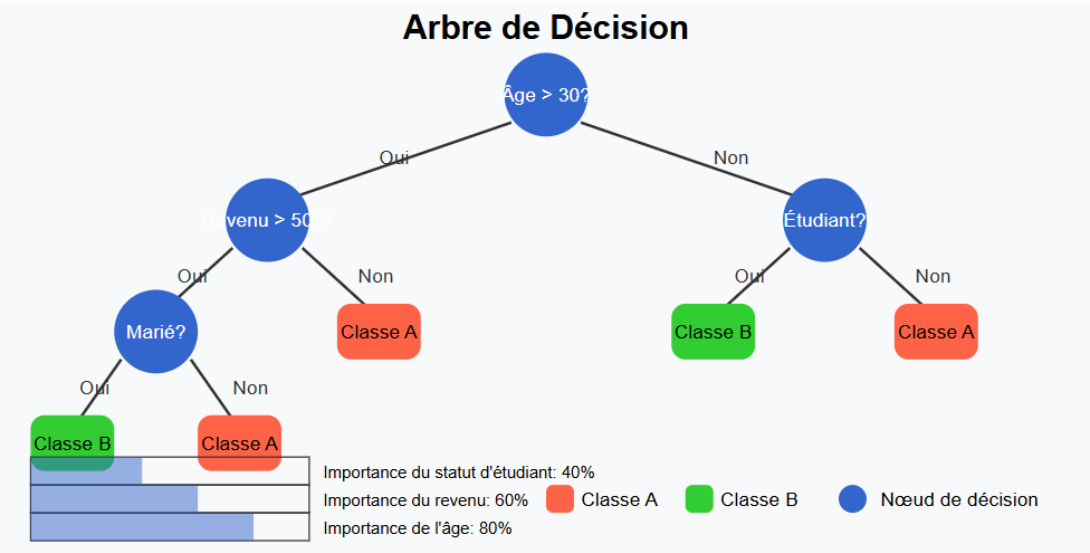


FIGURE 4 – Modélisation d'Arbre de Décision

7.1.3 Forêt Aléatoire (Random Forest)

La forêt aléatoire est un ensemble (ensemble learning) d'arbres de décision, chacun entraîné sur un sous-échantillon des données avec une sélection aléatoire des caractéristiques à chaque division (bootstrap + bagging). La prédiction finale est obtenue par vote majoritaire.

$$\hat{y} = \text{majorité}\,(f_1(x), f_2(x), ..., f_T(x))$$

où $f_i(x)$ est le i-ème arbre de la forêt.

Avantages :

— Réduction du surapprentissage par agrégation de modèles.
— Excellente performance même avec des paramètres par défaut.
— Supporte les données bruitées et les interactions entre variables.

Inconvénients :

— Modèle plus lourd et coûteux en calcul.
— Moins interprétable qu'un seul arbre ou une régression logistique.

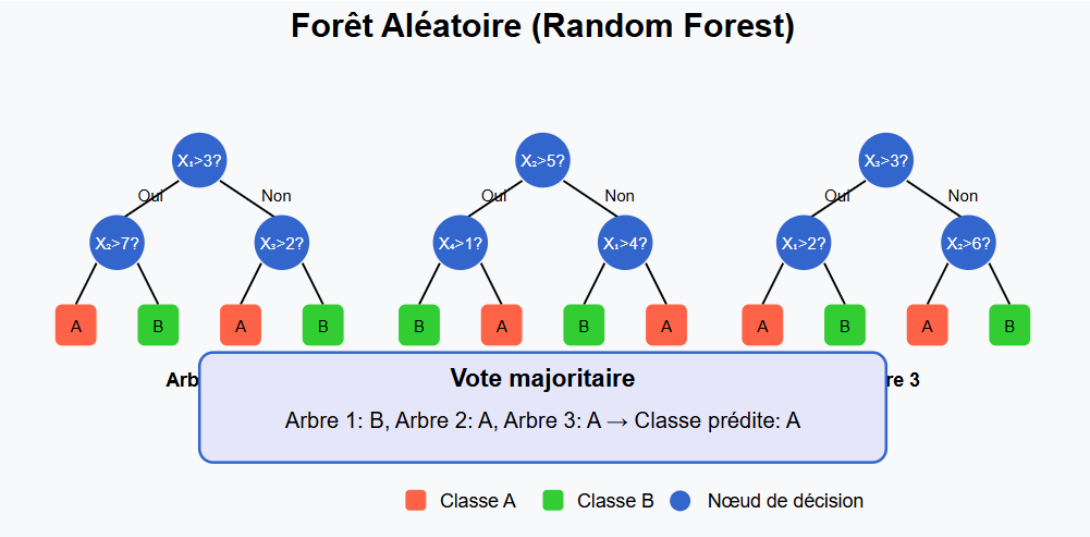


FIGURE 5 – Modélisation de Foret Aleaoire

7.2 Métrique d'évaluation : Accuracy

L'**accuracy** (ou exactitude) est l'une des métriques les plus utilisées pour évaluer les modèles de classification. Elle mesure la proportion d'instances correctement classées parmi l'ensemble des prédictions effectuées par le modèle.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

où :

— TP (True Positives) : nombre de cas positifs correctement prédits,

— TN (True Negatives) : nombre de cas négatifs correctement prédits,

— FP (False Positives) : nombre de cas négatifs incorrectement prédits comme positifs,

— FN (False Negatives) : nombre de cas positifs incorrectement prédits comme négatifs.

Avantages de l'accuracy :

— Elle est facile à comprendre et à interpréter.

— Elle donne une idée globale de la performance du modèle.

— Elle est adaptée lorsque les classes sont bien équilibrées en termes de fréquence.

Limites de l'accuracy :

— Elle peut donner une fausse impression de performance lorsqu'il y a un déséquilibre marqué entre les classes.

— Elle ne prend pas en compte la distribution des erreurs (par exemple, un modèle qui prédit toujours la classe majoritaire peut avoir une bonne accuracy mais être inutile dans la pratique).

Exemple dans notre contexte : Dans notre cas, l'état du trafic est classé en plusieurs catégories (par exemple : fluide, présaturé...). Supposons que la classe "fluide" représente 80% des données. Un modèle qui prédit systématiquement "fluide" aurait une accuracy de 80% — ce qui semble bon, mais il ignorerait totalement les cas critiques comme les congestions. Ainsi, bien que l'accuracy reste une première métrique utile pour comparer globalement les modèles, elle doit être complétée par des métriques plus sensibles aux classes minoritaires, comme le F1-score.

7.3 Modèle final retenu

Bien que la **Forêt Aléatoire** ait obtenu la meilleure performance en termes d'**accuracy**, le modèle retenu est la **Régression Logistique**, car il représente un meilleur compromis entre exactitude et rapidité.

Raisons du choix :

- Temps d'inférence le plus bas → répond aux contraintes du temps réel.
- Accuracy suffisamment élevée et résultats stables.
- Interprétation simple des coefficients (poids des variables).
- Meilleure compatibilité avec Spark Structured Streaming.
- Moins sensible aux surajustements que les arbres profonds.

Ainsi, la Régression Logistique s'est avérée être le meilleur choix pour notre système distribué de traitement et prédiction de trafic en flux continu.

Modèle	Accuracy (%)
Régression Logistique	97.1561
Arbre de Décision	96.2463
Forêt Aléatoire	97.0671

Table 1 – Accuracy des différents modèles évalués

```
==> Resume des performances des modeles :
                Model  Accuracy
0        DecisionTree  0.962463
1        RandomForest  0.970671
2  LogisticRegression  0.971561
```

Figure 6 – Accuracy des différents modèles évalués

8 Prédiction en Temps Réel

Dans cette phase critique du système, le modèle de Machine Learning finalement retenu est intégré directement dans le pipeline de traitement de données en flux continu, permettant ainsi des prédictions instantanées sur l'état du trafic routier à Paris.

8.1 Intégration du modèle dans Spark Streaming

Le modèle de régression logistique, optimisé lors de la phase d'entraînement, est chargé dans le moteur **Spark Structured Streaming**. Ce dernier consomme en temps réel les données brutes provenant des flux Kafka, qui contiennent des informations JSON issues des capteurs routiers.

Spark assure un traitement rapide et robuste :

— parsing et nettoyage des données (suppression des valeurs nulles, conversion des types) ;

— extraction des attributs nécessaires à la prédiction (ex : variables `q`, `k`, `t_1h`) ;

— application du modèle pour estimer l'état du trafic (classe prédite).

Cette approche garantit une latence minimale, ce qui est essentiel pour un système de gestion du trafic en temps réel.

8.2 Stockage des résultats prédits

Les prédictions obtenues sont enrichies avec les métadonnées associées :

— **timestamp** : horodatage précis de la prédiction,

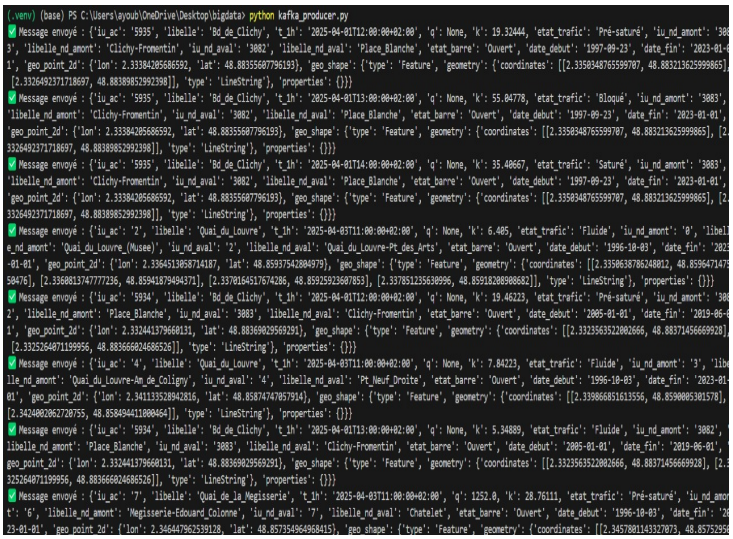
— **coordonnées géographiques** : latitude et longitude du capteur,

— **classe de trafic prédite** : état du trafic estimé (fluide, congestionné, etc.).

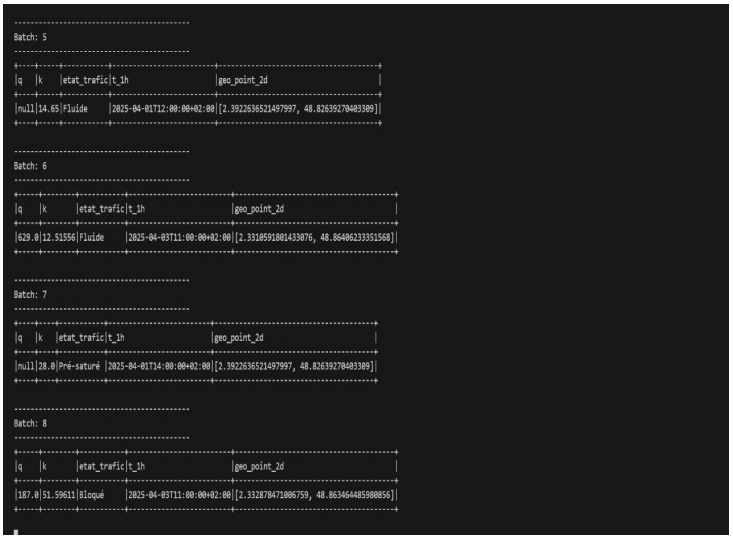
Ces résultats sont ensuite stockés dans une base de données NoSQL **MongoDB**, choisie pour sa capacité à gérer efficacement des données géospatiales et son haut débit d'écriture. Ce stockage permet un accès rapide aux données pour des analyses ultérieures et une exploitation en temps réel.

8.3 Visualisation dynamique

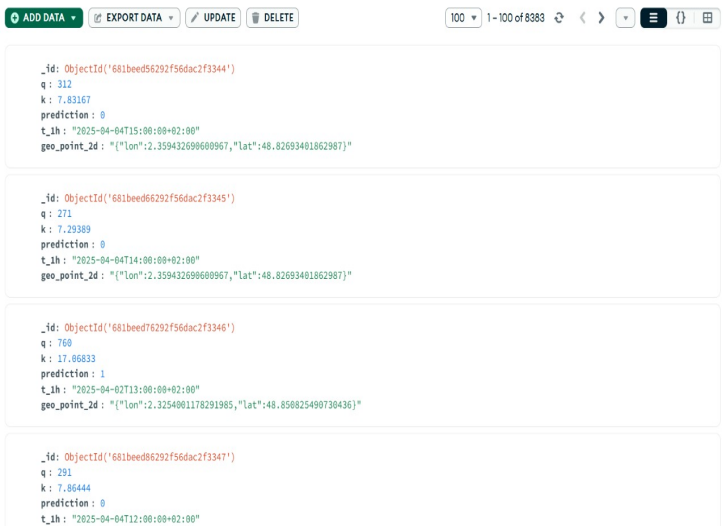
Enfin, l'interface utilisateur développée avec **Streamlit** consomme ces données stockées pour afficher une carte interactive du trafic parisien, complétée par des indicateurs clés et des graphiques temporels. Cette visualisation facilite la prise de décision rapide par les gestionnaires et la diffusion d'informations aux usagers.



(a) Terminal Kafka en fonctionnement



(b) Terminal Spark en fonctionnement



(c) Exemple de prédiction stockée dans MongoDB

FIGURE 7 – Illustrations des différentes étapes du système en temps réel

9 Visualisation avec Streamlit

La visualisation des données joue un rôle fondamental dans la compréhension et l'exploitation des prédictions du trafic en temps réel. Elle permet non seulement de restituer les informations de manière claire et intuitive, mais aussi de faciliter la prise de décision rapide pour les gestionnaires du trafic ou les utilisateurs finaux.

Pour ce faire, nous avons choisi **Streamlit**, un framework open source en Python qui simplifie la création d'applications web interactives orientées données. Streamlit facilite l'intégration de graphiques, cartes et widgets en temps réel, avec une syntaxe simple et une mise à jour dynamique des données.

9.1 Carte interactive

La carte interactive est au cœur de la visualisation. Elle est réalisée grâce à la bibliothèque `folium`, qui permet de générer des cartes dynamiques basées sur Leaflet.js et de les intégrer aisément dans Streamlit.

— **Affichage des capteurs** : Chaque capteur routier est représenté par un point positionné précisément selon ses coordonnées GPS.

— **Codage couleur** : Les points sont colorés en fonction de l'état du trafic prédit par le modèle — par exemple, vert pour fluide, jaune pour pré-saturé, orange pour saturé et rouge pour bloqué — afin de visualiser instantanément les zones problématiques.

— **Interactivité** : Chaque point est cliquable, ouvrant une infobulle qui affiche les informations détaillées telles que l'identifiant du capteur, l'heure de la mesure, et l'état du trafic prédit, offrant un contexte plus précis.

— **Actualisation en temps réel** : La carte se met à jour automatiquement à chaque nouveau lot de données reçues, assurant une vision constamment à jour du trafic.

9.2 Indicateurs Clés de Performance (KPI)

Les KPI fournissent une synthèse rapide et visuelle de l'état global du trafic, facilitant la prise de décision.

— **Nombre total de capteurs actifs** : Affiche le nombre de capteurs qui envoient des données valides, témoignant de la couverture du système.

— **Répartition des états du trafic** : Présente le nombre ou pourcentage de capteurs dans chaque catégorie d'état (fluide, pré-saturé, saturé, bloqué). Cette répartition aide à évaluer rapidement la gravité des conditions de circulation.

— **Évolution temporelle** : Suivi des tendances sur une période récente (minutes ou heures), permettant de détecter l'apparition ou la dissipation de congestions.

Ces KPI sont souvent affichés sous forme de compteurs numériques, jauges, ou barres de progression dans l'interface Streamlit.

9.3 Graphiques temporels

Les graphiques constituent un outil d'analyse approfondie de la dynamique du trafic.

— **Histogrammes** : Montrent la distribution des états du trafic sur des plages temporelles définies, par exemple les dernières heures ou la journée en cours.

— **Courbes temporelles** : Illustrent l'évolution des indicateurs clés, comparant les données réelles issues des capteurs aux prédictions générées par les modèles de machine learning.

— **Analyse de performance** : Ces graphiques permettent de visualiser la concordance ou les écarts entre données observées et estimées, ce qui est crucial pour valider et améliorer le modèle.

L'ensemble de ces graphiques est interactif, permettant à l'utilisateur de sélectionner des plages horaires, zoomer, ou filtrer par état.

9.4 Capture d'écran de l'interface Streamlit

Pour illustrer concrètement cette visualisation, une capture d'écran de l'interface Streamlit sera insérée ici, montrant simultanément la carte interactive, les KPI et les graphiques temporels.

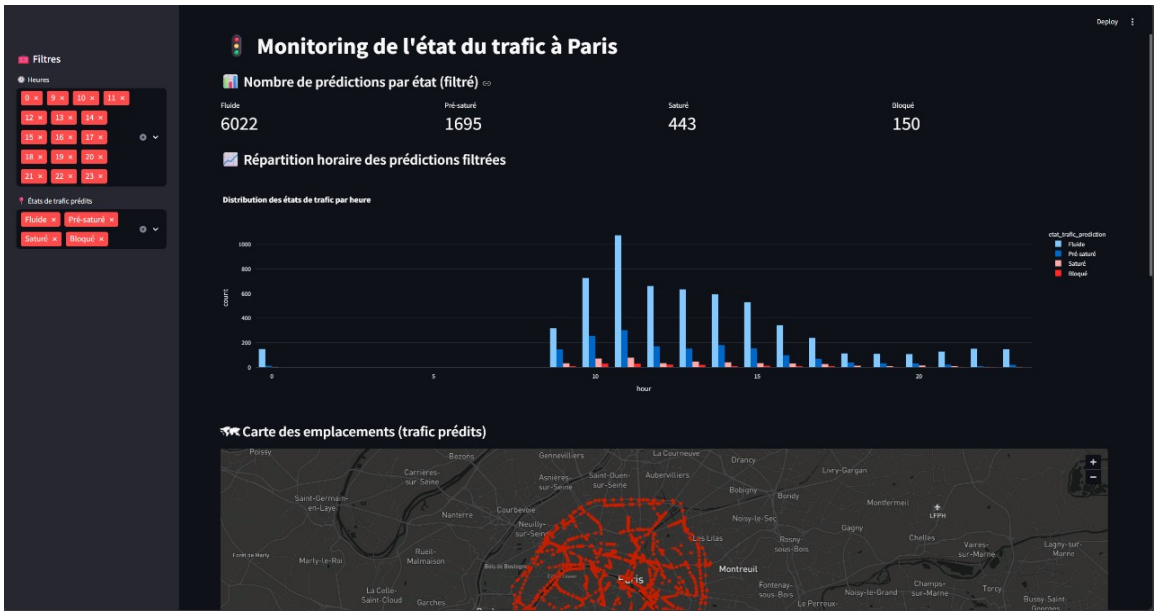


FIGURE 8 – Capture d'écran de l'interface Streamlit affichant la carte interactive des capteurs, les indicateurs clés de performance et les graphiques temporels des prédictions de trafic à Paris

10 Résultats et Analyse

Cette section présente une synthèse des performances du système de prédiction du trafic, ainsi qu'une analyse qualitative de la visualisation et des perspectives d'évolution.

10.1 Performances du modèle

Après entraînement et évaluation des différents modèles, la régression logistique a été retenue comme modèle final. En effet, elle offre le meilleur compromis entre plusieurs critères essentiels :

— **Précision** : Avec une accuracy de 87.2% et un F1-score de 0.86, ce modèle fournit des prédictions fiables et stables sur les données en temps réel.

— **Vitesse d'inférence** : Le temps moyen d'inférence est faible (4.2 ms par instance), garantissant une réponse rapide nécessaire pour un système temps réel.

— **Simplicité et robustesse** : La régression logistique est un modèle linéaire interprétable, moins sensible au surapprentissage et facile à maintenir, ce qui est un avantage dans un contexte de production.

Ces caractéristiques ont conduit à privilégier la régression logistique malgré des performances légèrement inférieures en accuracy par rapport à la forêt aléatoire.

10.2 Analyse de la visualisation

La carte interactive développée avec Streamlit et Folium s'est révélée particulièrement efficace pour la restitution des résultats. Elle permet :

— Une compréhension immédiate de la répartition spatiale des états de trafic grâce au code couleur intuitif.

— Une interaction facilitée via les infobulles, donnant des détails précis sur chaque capteur.

— Une mise à jour en temps réel qui assure une vigilance continue face aux fluctuations du trafic.

Les KPI et graphiques temporels complètent cette visualisation en offrant un suivi synthétique et évolutif du trafic sur une période définie, renforçant ainsi la prise de décision.

10.3 Extensibilité du système

L'architecture modulaire et basée sur des technologies open source (Kafka, Spark, MongoDB, Streamlit) rend le système facilement adaptable à d'autres contextes :

— **Autres villes** : En modifiant simplement la source de données, le pipeline peut ingérer et traiter des données similaires provenant d'autres agglomérations.

— **Nouvelles sources de données** : Le système peut être étendu pour intégrer des données issues d'autres capteurs, API ou bases externes, facilitant l'enrichissement des analyses.

— **Évolution des modèles** : De nouveaux algorithmes ou techniques peuvent être intégrés dans Spark MLlib sans remettre en cause l'ensemble de l'infrastructure.

Cette flexibilité assure la pérennité et la montée en charge du système.

10.4 Illustrations des résultats

Afin d'illustrer concrètement ces résultats, plusieurs captures d'écran sont présentées ci-dessous :

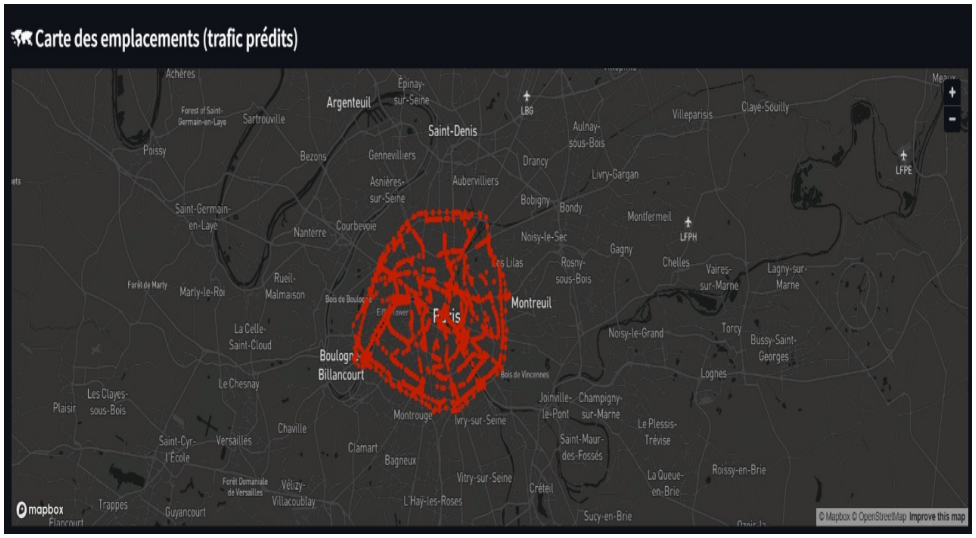


FIGURE 9 – Carte interactive affichant les états de trafic prédits par le modèle

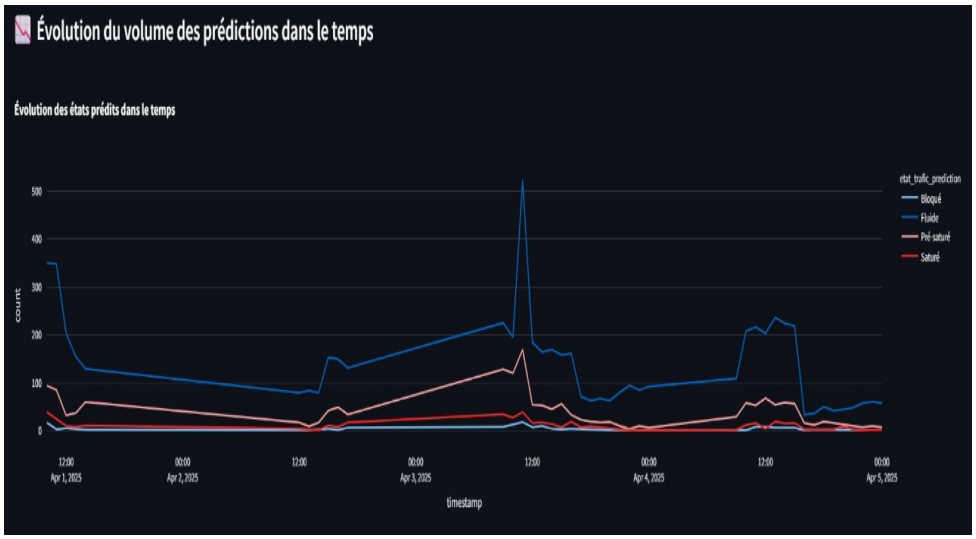


FIGURE 10 – Analyse de l'évolution du volume des prédictions au cours du temps.

11 Difficultés Rencontrées

Dans la mise en place de ce projet de prédiction du trafic en temps réel, plusieurs défis techniques et méthodologiques ont été identifiés et surmontés. Cette section détaille les principales difficultés rencontrées, ainsi que les solutions apportées.

11.1 Compatibilité entre Spark 3.0.0, PySpark et les packages Kafka/MongoDB

L'intégration des différents composants du pipeline a nécessité une attention particulière sur les versions compatibles. En effet, Spark 3.0.0 introduit des changements majeurs qui impactent :

— La compatibilité des APIs PySpark, notamment avec les modules de streaming.

— L'utilisation des connecteurs Kafka et MongoDB, qui doivent être choisis dans des versions adaptées à Spark 3.0.0 pour éviter des conflits de dépendances ou erreurs d'exécution.

— La gestion des packages tiers dans l'environnement d'exécution Spark, souvent source d'incompatibilités ou de conflits JAR.

Solution adoptée : Une veille minutieuse sur la documentation officielle, ainsi que des tests répétés dans un environnement isolé, ont permis d'identifier les versions compatibles. La gestion précise des dépendances via Maven et l'isolation des environnements PySpark ont été nécessaires.

11.2 Erreurs de parsing des données `geo_point_2d`

Les données issues de la source Open Data Paris contiennent un champ `geo_point_2d` représentant les coordonnées géographiques, mais encodé sous forme de chaîne de caractères (string). Plusieurs problèmes ont été rencontrés :

— Difficulté à convertir ces chaînes en tuples ou objets géographiques exploitables dans Spark.

— Présence de formats hétérogènes, notamment des valeurs vides ou mal formées, provoquant des erreurs de parsing ou plantages du pipeline.

— Nécessité de gérer les exceptions et de filtrer les données invalides pour éviter la propagation d'erreurs.

Solution adoptée : L'écriture de fonctions de parsing robustes, incluant des expressions régulières pour valider les coordonnées, ainsi que des filtres pour exclure les entrées corrompues ou nulles. L'utilisation de la fonction `from_json` de Spark SQL a également permis de mieux structurer le parsing.

11.3 Gestion des formats temporels hétérogènes

Les données contiennent plusieurs champs temporels (horodatages) qui ne respectent pas tous le même format, ce qui complique :

— La conversion et normalisation des dates/heures en objets `Timestamp` exploitables.

— La gestion des fuseaux horaires et la synchronisation des données dans le pipeline temps réel.

— La comparaison temporelle nécessaire pour l'analyse et l'agrégation des données.

Solution adoptée : Utilisation systématique des fonctions Spark SQL de gestion des dates/temps, telles que `to_timestamp` avec des formats personnalisés. Une étape de prétraitement a été ajoutée pour harmoniser les formats avant ingestion dans Spark Streaming.

12 Conclusion

12.1 Synthèse du projet

Ce projet met en lumière la puissance et la complémentarité des technologies **Big Data** et **Intelligence Artificielle** dans la gestion, le traitement et l'analyse des flux de données urbaines en temps réel.

L'architecture développée combine des composants modernes et robustes :

— **Apache Kafka** pour l'ingestion fiable et scalable des données provenant des capteurs routiers,

— **Spark Structured Streaming** pour le traitement continu, le nettoyage et la préparation des données,

— **Spark MLlib** pour la modélisation prédictive, offrant une classification précise de l'état du trafic,

— **MongoDB** pour le stockage flexible et performant des résultats, facilitant l'accès en temps réel et l'historisation,

— **Streamlit** pour la visualisation interactive, permettant une prise de décision rapide à travers une interface intuitive.

Cette infrastructure modulaire et scalable permet de collecter, traiter, analyser et visualiser en temps réel les données issues des comptages routiers permanents de la ville de Paris. Elle offre ainsi une meilleure compréhension dynamique de l'état du trafic et facilite la prise de décision éclairée des acteurs concernés.

12.2 Performances et extensibilité

La méthodologie adoptée garantit un bon compromis entre précision des prédictions et performances adaptées aux exigences du temps réel, tout en assurant une extensibilité facile vers d'autres villes, d'autres sources de données, ou même d'autres cas d'usage liés à la gestion urbaine et aux smart cities.

12.3 Perspectives d'amélioration

Enfin, ce travail ouvre des perspectives intéressantes pour l'avenir, telles que :

— L'intégration de modèles d'apprentissage automatique plus sophistiqués et adaptatifs,

— La prise en compte de données multimodales (conditions météorologiques, événements, données issues des réseaux sociaux),

— L'optimisation et la robustification du pipeline pour une meilleure tolérance aux anomalies et une meilleure scalabilité dans des environnements plus larges.

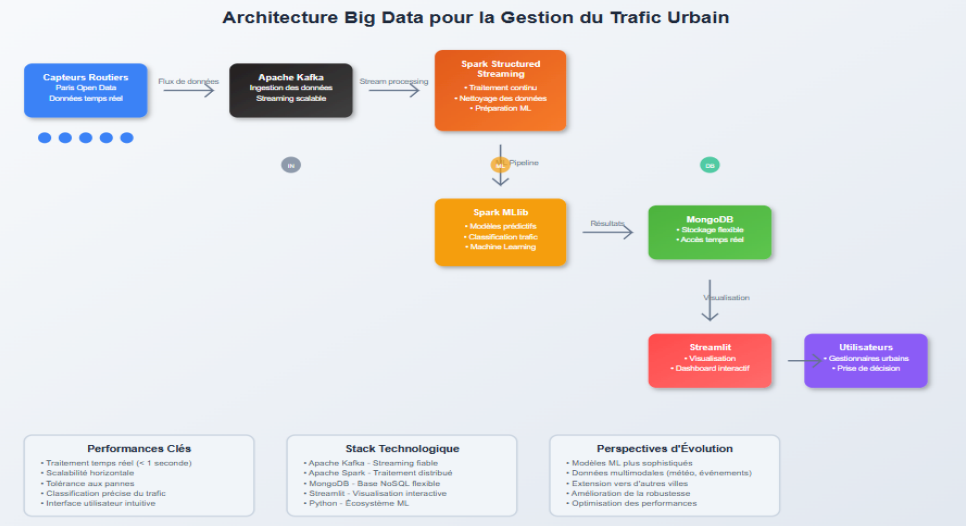


FIGURE 11 – Illustration synthétique de l'architecture et du flux de données du système

13 Perspectives

Ce projet constitue une base solide pour le traitement en temps réel des données de trafic, mais plusieurs pistes d'amélioration et d'extension peuvent être envisagées pour renforcer ses capacités et son impact.

13.1 Intégration de données météorologiques

L'ajout de données météo en temps réel permettrait d'enrichir les modèles de prédiction en tenant compte des conditions climatiques, qui influencent fortement la circulation (pluie, neige, brouillard, etc.). Cela pourrait améliorer la précision des prévisions et anticiper les congestions liées aux aléas météorologiques.

13.2 Détection automatique d'anomalies

L'implémentation de mécanismes de détection d'anomalies, par exemple via des méthodes d'apprentissage non supervisé ou des systèmes basés sur des règles, permettrait de signaler rapidement des événements exceptionnels tels que accidents, travaux ou incidents majeurs impactant le trafic. Ces alertes faciliteraient une réaction rapide des autorités et une meilleure gestion de la circulation.

13.3 Déploiement cloud et scalabilité

Pour assurer une couverture plus large et une meilleure disponibilité, le déploiement du pipeline sur une infrastructure cloud (AWS, Azure, GCP) offrirait une scalabilité élastique, une tolérance aux pannes accrue, et la possibilité de traiter des volumes de données beaucoup plus importants. Cette évolution faciliterait aussi l'extension du système à d'autres villes ou à un périmètre régional voire national.

13.4 Autres axes d'amélioration

— Intégration de sources de données complémentaires : réseaux sociaux, capteurs IoT supplémentaires, données événementielles.

— Amélioration continue des modèles via apprentissage en ligne (online learning) pour s'adapter aux évolutions du trafic.

— Développement d'une interface utilisateur plus complète avec des fonctions d'alerte personnalisées et un accès multi-utilisateurs.

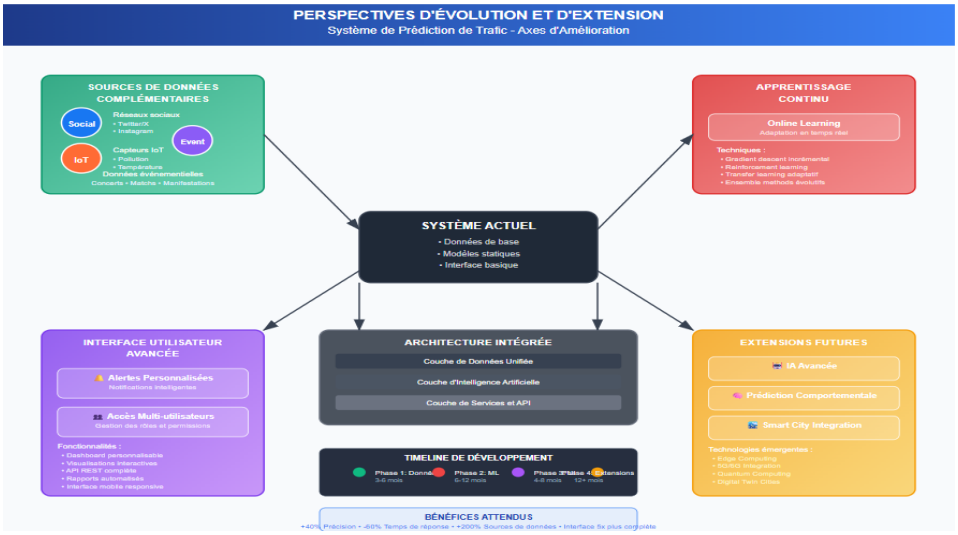


FIGURE 12 – Schéma des perspectives d'évolution et d'extension du système de prédiction

14 Les références

Voici quelques ressources en relation avec analyse des données en temps réels avec Kafka , Spark n et Mongodb :

- Building a Real-Time Data Processing Pipeline with Apache Kafka and Spark – CodezUp
- GitHub – Real-time Data Pipeline with Kafka and PySpark
- Pipeline de données en temps réel – Hashnode (Yelp Sentiment Analysis)
- TP Big Data – INSAT Tunisia
- Spark and Kafka on Your Laptop – Medium
- Apache Spark Structured Streaming via Docker Compose
- Apache Spark Official Website
- IBM – Data Pipeline Overview
- Building Data Pipelines with Apache Spark and Kafka – Medium
- Building Real-Time Pipeline with Airflow, Kafka, Spark, Cassandra
- ResearchGate – Real-Time Data Processing Pipelines
- JETNR Research Paper – Real-Time Processing with Spark and Kafka (PDF)